LLM Fundamentals: Architecture, Training & Inference

IDS 576: Deep Learning and Modern Applications

Dr. Theja Tulabandhula

University of Illinois Chicago

Learning Objectives
Part I: Decoder-Only Transformers
Where GPT Fits in the Transformer Family
From Original Transformer to GPT
GPT Training Objective
Causal Masking
GPT-2 Architecture
Autoregressive Text Generation
Decoding Strategies
BERT vs GPT: Key Differences
Why GPT-2 (2019) Was Important
Part II: Training Large Language Models
The LLM Training Pipeline
Tokenization
Pretraining

Instruction Tuning

Reinforcement Learning from Human Feedback

Advanced Training Topics

The Modern LLM Stack

Part III: Optimizing LLM Inference

Inference Phases

KV Cache

Quantization

Batching Strategies

Optimized Runtimes

Performance Metrics

Summary

References

Learning Objectives

Learning Objectives

After this lecture, you will be able to:

- Distinguish between encoder-only, encoder-decoder, and decoder-only transformer architectures
- Explain the autoregressive training objective used in GPT models
- Understand how causal masking prevents attention to future tokens
- Describe the GPT-2 architecture and its components
- Compare different text generation strategies (greedy, sampling, top-k, top-p)
- Contrast bidirectional (BERT) vs causal (GPT) attention mechanisms
- Describe the complete LLM training pipeline from raw text to aligned model
- Explain Byte-Pair Encoding (BPE) tokenization and why it's preferred

Learning Objectives (cont.)

- Understand the scale and objectives of pretraining phases
- Distinguish between instruction tuning and RLHF alignment
- Explain how Proximal Policy Optimization (PPO) works in RLHF
- Understand Mixture-of-Experts (MoE) architecture and reasoning models
- Explain the two phases of LLM inference (prefill and decode)
- Understand why KV cache is critical for efficiency
- Describe how PagedAttention improves memory management
- Compare different quantization strategies and their trade-offs

Part I: Decoder-Only Transformers

Where GPT Fits in the Transformer Family

Three Major Transformer Designs

Encoder-only (bidirectional attention)

- BERT, RoBERTa
- Used for classification and representation learning
- Can see entire sequence simultaneously

Encoder-Decoder (cross-attention)

- T5, BART
- Used for sequence-to-sequence tasks
- Translation, summarization

Decoder-only (causal attention)

- GPT-2, GPT-3, GPT-4, LLaMA
- Used for text generation
- Foundation for modern LLMs

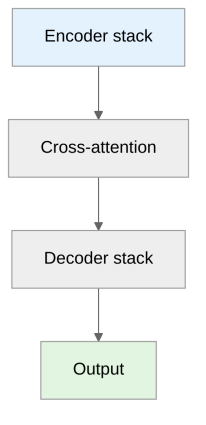
Architecture Comparison

Architecture	Attention Type	Primary Use	Examples
Encoder-only	Bidirectional	Understanding	BERT, RoBERTa
Encoder-Decoder	Bidirectional + Cross	Translation	T5, BART
Decoder-only	Causal	Generation	GPT, LLaMA

Key insight: Different attention mechanisms enable different capabilities.

From Original Transformer to GPT

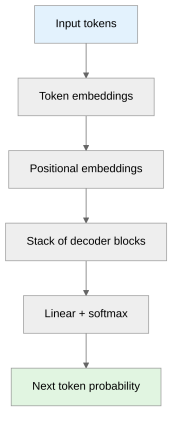
Original Transformer (2017)



- Designed for translation tasks
- Both encoder and decoder components
- Cross-attention connects them

GPT Simplification

Key idea: Remove the encoder entirely



Result: A **causal language model**

Why Decoder-Only Works

- Language modeling doesn't need encoding step
- Generation is inherently sequential
- Simpler architecture
- Scales efficiently
- Same mechanism for all tokens

Empirical finding: Decoder-only models match or exceed encoder-decoder performance on many tasks.

GPT Training Objective

Autoregressive Language Modeling

Training data example:

```
Input sequence:  "The cat sat on the"
Targets:         "cat sat on the mat"
```

The model learns:

$$P(w_t \mid w_1, w_2, \dots, w_{t-1})$$

Loss: Cross-entropy over vocabulary at each position

Training vs Inference

Training:

- All tokens processed in parallel
- Teacher forcing: use ground truth previous tokens
- Efficient on GPUs

Inference:

- Sequential generation
- Use model's own predictions
- One token at a time

Mathematical Formulation

Given sequence $w_1, \dots, w_n$, maximize:

$$\mathcal{L} = \sum_{t=2}^{n} \log P(w_t \mid w_1, \dots, w_{t-1})$$

- Each token prediction conditioned on all previous tokens
- No supervision for future tokens
- Self-supervised learning from raw text

Causal Masking

The Problem

Without constraints, self-attention can see **all** tokens:

```
Full Attention Matrix (no masking)

       t1    t2    t3    t4
t1  [ ✓     ✓     ✓     ✓ ]
t2  [ ✓     ✓     ✓     ✓ ]
t3  [ ✓     ✓     ✓     ✓ ]
t4  [ ✓     ✓     ✓     ✓ ]
```

This would allow "cheating" during training!

The Solution: Causal Mask

Attention mask matrix (lower triangular):

```
Causal Attention Matrix (lower triangular)

       t1    t2    t3    t4
t1  [ ✓     ✗     ✗     ✗ ]
t2  [ ✓     ✓     ✗     ✗ ]
t3  [ ✓     ✓     ✓     ✗ ]
t4  [ ✓     ✓     ✓     ✓ ]
```

The Solution: Causal Mask (cont.)

Meaning:

- Token 3 can attend to tokens 1, 2, 3
- Token 3 **cannot** attend to token 4 (future)

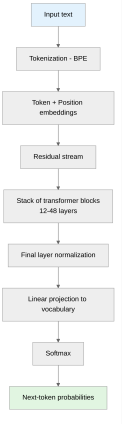
Why Causal Masking Matters

- Enforces left-to-right generation
- Prevents information leakage during training
- Ensures valid probability model
- Critical for autoregressive property

Key principle: Future tokens are hidden

GPT-2 Architecture

GPT-2 Architecture: Pipeline



GPT-2 Architecture: Pipeline (cont.)

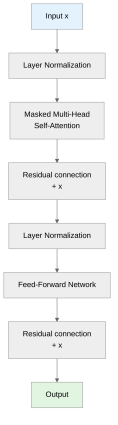
Key stages:

- Tokenization using Byte-Pair Encoding (BPE)
- Token + Positional embeddings
- Stack of decoder blocks (12-48 layers)
- Linear projection to vocabulary

Context length: 1024 tokens (GPT-2)

GPT-2 Decoder Block

Each layer contains:



GPT-2 Decoder Block (cont.)

Components:

- Layer Normalization (before attention)
- Masked Multi-Head Self-Attention
- Residual connections
- Feed-Forward Network

Note: Pre-LN architecture (layer norm before attention), unlike original transformer

GPT-2 Model Sizes

OpenAI released four variants:

Model	Parameters	Layers	Hidden Size
Small	124M	12	768
Medium	355M	24	1024
Large	774M	36	1280
XL	1.5B	48	1600

Key details:

- **Tokenizer**: Byte-Pair Encoding (BPE)
- **Vocabulary size**: ~50,257 tokens

GPT-2 Architectural Details

Positional embeddings:

- Learned (not sinusoidal)
- Maximum sequence length: 1024

Attention heads:

- 12 heads (small/medium)
- 16-25 heads (large/XL)

Feed-forward expansion:

- Hidden size $\times$ 4 dimensions
- GELU activation

Autoregressive Text Generation

Generation Loop

```
Step 1: "The capital of France is"       → Model → "Paris"
Step 2: "The capital of France is Paris"  → Model → "."
Step 3: "The capital of France is Paris." → Model → [EOS]
```

Core loop: predict $\rightarrow$ append $\rightarrow$ repeat

Generation Process Details

1. **Encode prompt** into token IDs
2. **Forward pass** through model
3. **Sample** next token from probability distribution
4. **Append** token to sequence
5. **Repeat** until:
    - End-of-sequence token generated
    - Maximum length reached
    - Custom stopping condition

Decoding Strategies

The Challenge

Model outputs a probability distribution over ~50k tokens:

```
Paris       0.45
Lyon        0.20
Marseille   0.10
London      0.05
Berlin      0.03
...
```

Question: How do we choose the next token?

Greedy Decoding

Strategy: Always pick highest probability token

```
next_token = argmax(probabilities)
```

Pros:

- Fast
- Deterministic
- Simple

Cons:

- Repetitive
- May miss better sequences
- No randomness

Temperature Sampling

Strategy: Scale logits before softmax

```
probabilities = softmax(logits / temperature)
next_token = sample(probabilities)
```

Temperature effects:

- $T < 1$: More confident (sharper distribution)
- $T = 1$: Unchanged
- $T > 1$: More random (flatter distribution)

Top-k Sampling

Strategy: Sample from k most likely tokens

```
top_k_probs, top_k_indices = topk(probabilities, k)
renormalize(top_k_probs)
next_token = sample(top_k_probs)
```

Typical k values: 40-50

Benefit: Prevents sampling very unlikely tokens while maintaining diversity

Top-p (Nucleus) Sampling

Strategy: Sample from smallest set with cumulative probability > p

```
sorted_probs = sort(probabilities, descending=True)
cumsum_probs = cumsum(sorted_probs)
nucleus = sorted_probs[cumsum_probs <= p]
next_token = sample(nucleus)
```

Typical p values: 0.9-0.95

Benefit: Adaptive cutoff based on distribution shape

Decoding Strategy Comparison

Strategy	Determinism	Quality	Speed	Use Case
Greedy	Fully	Repetitive	Fast	Extractive tasks
Temperature	Tunable	Variable	Fast	Creative writing
Top-k	Partial	Good	Medium	General purpose
Top-p	Partial	Excellent	Medium	High-quality generation

Common practice: Combine temperature + top-p

BERT vs GPT: Key Differences

Attention Mechanism

BERT (bidirectional):

```
token ↔ token
(can see all context)
```

GPT (causal):

```
token → token
(can only see previous)
```

Implication: BERT understands, GPT generates

Training Objective

BERT:

- Masked language modeling (Predict masked tokens)
- Next sentence prediction

GPT:

- Causal language modeling (Predict next token)
- No sentence-level task

Architectural Differences

Aspect	BERT	GPT
Type	Encoder-only	Decoder-only
Attention	Bidirectional	Causal
Position	Absolute learned	Absolute learned
Best for	Classification	Generation
Pre-LayerNorm	No	Yes (GPT-2+)

pre-LayerNorm: layer normalization is applied before the sub-layer (attention or FFN). Lead to more gradient stability.

When to Use Which?

BERT (and encoder models):

- Text classification
- Named entity recognition
- Question answering (extractive)
- Sentiment analysis

GPT (and decoder models):

- Text generation
- Completion tasks
- Dialogue systems
- Translation (with prompting)
- Few-shot learning

Why GPT-2 (2019) Was Important

Key Contributions

1. **Scale**: Demonstrated benefits of larger models
2. **Unsupervised pretraining**: Single objective, no task-specific architecture
3. **Transfer learning**: Zero-shot and few-shot capabilities
4. **Simplicity**: Decoder-only architecture sufficient

Scaling Laws Observation

As model size increased:

- Performance improved on downstream tasks
- Zero-shot capabilities emerged
- Better context understanding
- More coherent long-form generation

Insight: Scaling model size is a viable path to better performance

From GPT-2 to Modern LLMs

Same basic architecture powers (2023-2026):

- GPT-3 (175B parameters)
- GPT-4 (rumored 1.7T+ parameters, MoE)
- LLaMA (7B-70B)
- Mistral (7B)
- DeepSeek (multiple variants)

Changes mostly involve:

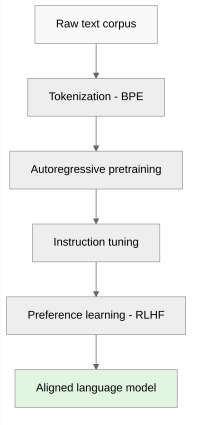
- Scale (parameters, data, compute)
- Training data composition
- Alignment methods (RLHF)
- Architectural tweaks (MoE, etc.)

Core mechanism remains: Decoder-only transformer

Part II: Training Large Language Models

The LLM Training Pipeline

Overview: Raw Text to Aligned Model



Each stage serves a distinct purpose in creating useful AI systems.

Why Multiple Stages?

Pretraining alone produces a model that:

- Completes text
- Predicts next tokens
- But doesn't follow instructions well
- May generate harmful content

Solution: Multi-stage training

1. **Pretraining**: Learn language patterns
2. **Instruction tuning**: Learn to follow commands
3. **RLHF**: Align with human values

Tokenization

Why Not Use Characters or Words?

Character-level:

- Pros: Small vocabulary, handles any text
- Cons: Very long sequences, hard to learn meaning

Word-level:

- Pros: Semantic units
- Cons: Huge vocabulary, can't handle rare words or typos

Subword tokenization: Best of both worlds

Byte-Pair Encoding (BPE)

Core idea: Start with characters, merge frequent pairs

Algorithm:

1. Initialize vocabulary with all characters
2. Find most frequent adjacent pair
3. Merge into single token
4. Repeat for desired vocabulary size

Result: Subword vocabulary balancing coverage and size

BPE Example

Original text: `"learning learning learned"`

Initial: `l e a r n i n g _ l e a r n i n g _ l e a r n e d`

Merge 1 (most frequent): `le` $\rightarrow$ `le arning le arning le arned`

Merge 2: `le + arning` $\rightarrow$ `learning learning learned`

Final vocabulary: {characters, `le`, `arning`, `learning`, ...}

BPE in Practice

GPT-2/GPT-3:

- Vocabulary size: ~50k tokens
- Handles:
  - Common words: single token
  - Rare words: multiple subwords
  - Any Unicode: fallback to bytes

Example tokenization:

```
"ChatGPT works well"
→ ["Chat", "G", "PT", " works", " well"]
```

Why BPE Matters

Benefits:

- Compact vocabulary (~50k vs millions of words)
- Handles rare words gracefully
- No out-of-vocabulary tokens
- Efficient for neural networks

Tradeoff:

- Adds preprocessing step
- Token boundaries may not align with semantic units
- Different languages may need different merges

Pretraining

Goal: Learn General Language Patterns

Objective: Predict next token

$$\mathcal{L} = -\sum_{t=2}^{T} \log P(w_t \mid w_{<t})$$

Training data: Massive unlabeled text (billions to trillions of tokens)

Duration: Weeks to months on thousands of GPUs

Pretraining Data Sources

Typical sources:

- Web pages (filtered for quality)
- Books
- Wikipedia
- Code repositories (GitHub)
- Research papers
- News articles

Preprocessing:

- Deduplication
- Quality filtering
- Toxicity filtering (though imperfect)
- Format standardization

Scale of Pretraining

Example datasets:

- GPT-3: ~300B tokens (~500GB text)
- LLaMA: ~1.4T tokens
- GPT-4 (rumored): Multiple trillions of tokens

Computational cost:

- Thousands of GPUs
- Multiple weeks of training
- Millions of dollars

Outcome: Base model with strong language understanding

Instruction Tuning

The Problem with Base Models

Base model behavior:

```
Prompt: "Summarize this article:"
Output: "Here are some related articles..."
```

- Completes text, doesn't follow instructions
- Not helpful for users
- Needs task-specific guidance

Supervised Fine-Tuning (SFT)

Approach: Train on (prompt, ideal response) pairs

Dataset format:

```
{
  "prompt": "Explain photosynthesis in simple terms",
  "response": "Photosynthesis is how plants make food
               using sunlight, water, and carbon dioxide..."
}
```

Dataset size: Typically 10k-100k high-quality examples

SFT Data Collection

Sources:

- Human-written demonstrations
- Curated from existing datasets
- Synthetic generation from stronger models
- Crowdsourced labeling

Quality over quantity: Better to have fewer high-quality examples

SFT Training Details

Objective: Standard supervised learning

$$\mathcal{L}_{\text{SFT}} = -\sum_i \log P(\text{response}_i \mid \text{prompt}_i)$$

Hyperparameters:

- Lower learning rate than pretraining
- Few epochs (1-3)
- Careful to avoid catastrophic forgetting

Outcome: Model that follows instructions

Reinforcement Learning from Human Feedback

RLHF: Aligning with Human Preferences

Goal: Make model responses:

- Helpful (addresses user intent)
- Harmless (avoids harmful content)
- Honest (truthful, acknowledges uncertainty)

Challenge: Hard to specify these properties in supervised way

Solution: Learn from comparative preferences

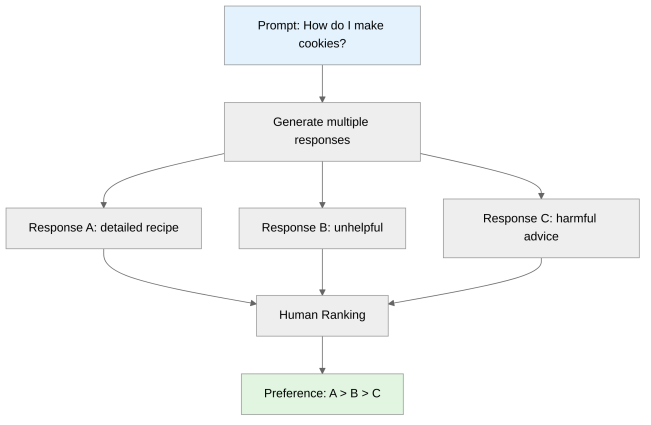
RLHF Stage 1: Collect Rankings

Process:

1. Generate multiple responses for same prompt
2. Human annotators rank responses
3. Build preference dataset

RLHF Stage 1: Collect Rankings (cont.)

Example:



RLHF Stage 2: Train Reward Model

Objective: Predict human preferences

Architecture: Same as LLM, but output is scalar score

Training:

$$\mathcal{L}_{\text{reward}} = -\mathbb{E}_{(x,y_w,y_l)}[\log \sigma(\tilde{r}(x, y_w) - \tilde{r}(x, y_l))]$$

Where:

- y_w = preferred (winner) response
- y_l = less preferred (loser) response
- $\tilde{r}$ = reward model

Outcome: Reward model that scores responses

RLHF Stage 3: RL Optimization

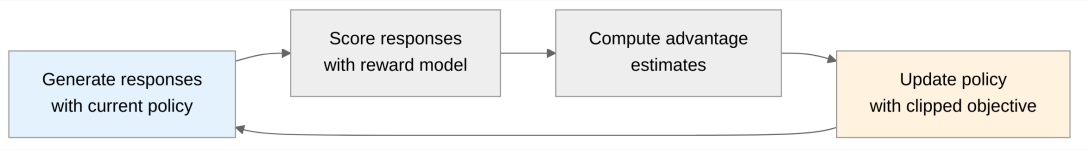
Setup:

- **Agent**: Language model (policy)
- **Environment**: Prompt distribution
- **Action**: Generate response
- **Reward**: Score from reward model

Constraint: Don't drift too far from SFT model (KL penalty)

Proximal Policy Optimization (PPO)

PPO loop:



Key innovation: Clip policy updates to prevent instability

$$\mathcal{L}_t^{\text{CLIP}} = \mathbb{E}[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)]$$

Where:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ = probability ratio (new vs old policy)
- $\hat{A}_t$ = estimated advantage (how much better than expected)
- ϵ = clipping range (typically 0.1–0.2)

RLHF Challenges

Reward hacking:

- Model exploits reward model weaknesses
- Generates responses that score high but aren't good

Training instability:

- RL can be unstable
- Need careful hyperparameter tuning
- KL penalty prevents mode collapse

Computational cost:

- Need both policy and reward model in memory
- Multiple generations per step
- Slower than supervised training

Advanced Training Topics

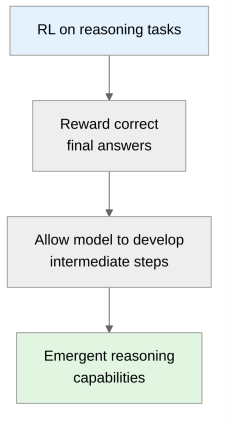
Reasoning Models

Recent innovation (OpenAI o1, DeepSeek-R1):

- Emphasize multi-step reasoning
- Chain-of-thought generation
- Self-verification capabilities

Reasoning Models (cont.)

Training approach:



Mixture-of-Experts (MoE)

Standard transformer: All parameters active for every token

MoE transformer: Route each token to subset of experts

```
Input token
    ↓
Router network
    ↓
Select top-k experts (e.g., 2 out of 8)
    ↓
Combine expert outputs
    ↓
Output
```

Benefit: Scale model capacity without proportional compute increase

MoE Architecture Details

Components:

- **Router**: Learned gating network (usually softmax over experts)
- **Experts**: Independent feed-forward networks
- **Combining**: Weighted sum of expert outputs

Parameters:

- Total: 671B (DeepSeek-V3)
- Active per token: ~37B

Training: Router and experts trained jointly

Why MoE Works

Intuition: Different tokens need different processing

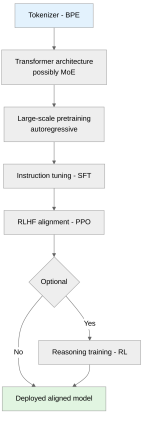
- Code tokens $\rightarrow$ code-specialized expert
- Math tokens $\rightarrow$ math-specialized expert
- General text $\rightarrow$ general expert

Empirical finding: Models learn meaningful expert specialization

Tradeoff: More complex training, potential load-balancing issues

The Modern LLM Stack

Putting It All Together



This pipeline defines most production LLM systems.

Training Timeline

Typical frontier model:

1. **Pretraining**: 1-3 months on thousands of GPUs
2. **Instruction tuning**: Days to weeks on hundreds of GPUs
3. **RLHF**: Weeks on hundreds of GPUs
4. **Total**: 2-6 months, millions of dollars

Smaller models: Can be done in days/weeks with fewer resources

What Makes Modern LLMs Work

It's not one thing, but the combination:

- Architecture (transformer, MoE)
- Scale (parameters, data, compute)
- Training pipeline (pretraining $\rightarrow$ SFT $\rightarrow$ RLHF)
- Data quality (filtering, curation)
- Compute infrastructure (distributed training)

Result: Models that are helpful, harmless, and capable

Part III: Optimizing LLM Inference

Inference Phases

Overview

Key challenges in LLM inference:

- High latency for token generation
- GPU memory constraints
- Low throughput with sequential generation
- Cost of serving at scale

Important optimization techniques:

- KV cache for attention reuse
- Batching strategies
- Model and KV cache quantization
- Optimized inference runtimes (vLLM, TensorRT-LLM)

Two-Phase Generation

LLM inference consists of two distinct phases:

Prefill Phase:

```
Process entire prompt
Compute attention for all prompt tokens
Store keys and values in cache
Generate first token
```

Decode Phase

```
Generate tokens one-by-one
Reuse cached keys/values from prompt
Append new token's K/V to cache
Repeat until end token or max length
```

Performance characteristics differ significantly between phases

Phase Characteristics

Aspect	Prefill	Decode
Computation	Parallel (all prompt tokens)	Sequential (one token)
Memory	Initial KV cache allocation	Growing KV cache
Bottleneck	Compute-bound	Memory-bound
Optimization	Batching, quantization	KV cache efficiency

Key insight: Most serving latency comes from the decode phase for long sequences

KV Cache

What is KV Cache?

Transformer attention computes:

```
Attention(Q, K, V) = softmax(QK^T / √d_k) V
```

Without cache: Recompute K and V for all previous tokens at each step

With cache: Store computed K and V tensors, only compute for new token

KV Cache: Memory Requirements

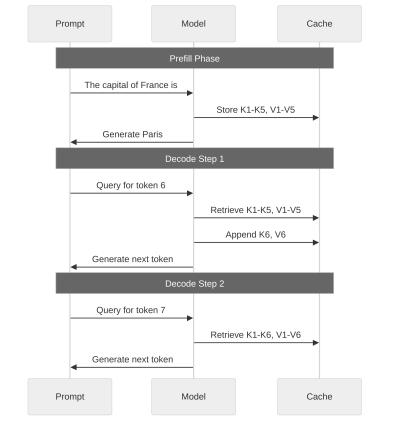
Memory per token:

`2 × n_layers × d_model × (FP16 or FP8)`

For `GPT-3 (175B)`:

- 96 layers, 12,288 dimension
- ~3.2 MB per token (FP16)
- For 2048 context: ~6.5 GB just for KV cache

KV Cache Diagram



Result: O(1) work per new token instead of O(n)

Memory Management Challenge

KV cache grows with:

- Sequence length (linear)
- Batch size (linear)
- Number of layers (linear)

Example calculation (7B model):

```
32 layers × 4096 dim × 2 (K+V) × 2 bytes (FP16)
= 524 KB per token

For batch size 32, context 2048:
= 32 × 2048 × 524 KB = 34 GB
```

This can exceed available GPU memory!

Solutions: PagedAttention (vLLM)

Insight: Treat KV cache like virtual memory in operating systems

Key ideas:

1. Split KV cache into fixed-size blocks (pages)
2. Store blocks non-contiguously in memory
3. Use block table to track logical-to-physical mapping
4. Share blocks across requests (prefix caching)

Benefits:

- Near-zero memory waste
- Enables dynamic batching
- Allows efficient memory sharing

Implementation: `vLLM`

Quantization

Model Weight Quantization

Idea: Store weights with fewer bits

Common formats:

```
FP32 → FP16 (2× reduction)
FP16 → INT8 (2× reduction)
FP16 → INT4 (4× reduction)
```

Trade-offs:

- Smaller memory footprint
- Faster computation on specialized hardware
- Potential accuracy degradation

Popular Quantization Methods

- `GPTQ`: Post-training quantization
- `AWQ`: Activation-aware quantization
- `SmoothQuant`: Joint activation-weight quantization

KV Cache Quantization

Challenge: KV cache is also memory-intensive

Solution: Quantize cached activations

```
FP16 → FP8 (2× reduction)
FP16 → INT8 (2× reduction)
```

Benefits:

- Enables longer context windows
- Supports larger batch sizes
- Minimal quality degradation with proper calibration

Example: FP8 KV cache allows 2× larger batch size or 2× longer context

Quantization Trade-offs

Method	Memory	Speed	Quality
FP16 (baseline)	1×	1×	100%
FP8 weights + cache	2×	1.5-2×	98-99%
INT8 weights	2×	2-3×	95-98%
INT4 weights	4×	3-4×	90-95%

Best practices:

- Use FP8/INT8 for production (minimal quality loss)
- INT4 useful for edge deployment with careful calibration
- Always benchmark on your specific tasks

Batching Strategies

Static Batching

Traditional approach:

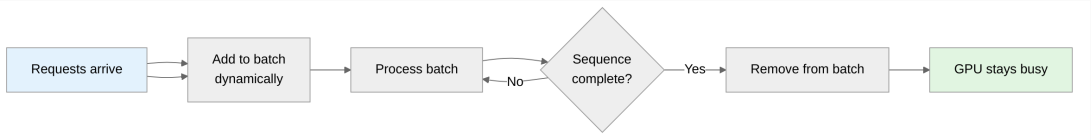
```
Wait for batch_size requests
Process entire batch together
Wait for all sequences to complete
```

Problems:

- Wasted GPU cycles waiting for batch to fill
- Stragglers delay entire batch
- Fixed batch size inefficient for variable-length sequences

Continuous Batching

Modern approach (used by vLLM, TensorRT-LLM):



Continuous Batching: Benefits

- Higher GPU utilization
- Lower latency (no waiting for batch to fill)
- Better throughput (no stragglers)

Key innovation: Efficient KV cache management (PagedAttention)

Benchmark results: 2-3$\times$ higher throughput vs static batching

Optimized Runtimes

vLLM

Key features:

- PagedAttention for memory efficiency
- Continuous batching
- Optimized CUDA kernels
- Support for various quantization formats

Performance improvements:

- 2-24$\times$ higher throughput vs HuggingFace Transformers
- Near-zero memory waste

vLLM: Usage Example

```
from vllm import LLM, SamplingParams

llm = LLM(model="meta-llama/Llama-2-7b-hf")
outputs = llm.generate(prompts, sampling_params)
```

Repository: `vllm-project/vllm`

Other Inference Engines

TensorRT-LLM (NVIDIA):

- Optimized for NVIDIA GPUs
- In-flight batching
- FP8/INT8/INT4 quantization
- Multi-GPU tensor parallelism

Text Generation Inference (TGI) by HuggingFace:

- Production-ready serving
- Built-in safeguards and monitoring
- Dynamic batching
- Streaming support

Edge and CPU Inference

llama.cpp:

- CPU-focused inference
- Quantization (4-bit, 5-bit, 8-bit)
- No GPU required
- Excellent for edge deployment

Performance Metrics

Key Metrics for Inference

Latency metrics:

- **Time to First Token (TTFT):** Prefill latency
- **Inter-Token Latency (ITL):** Time between tokens (decode)
- **Total latency:** End-to-end request completion

Throughput metrics:

- **Tokens per second (tok/s):** Overall generation rate
- **Requests per second:** Number of concurrent requests served

Resource Metrics

- **GPU memory usage:** Peak and average
- **GPU utilization:** Percentage of compute used
- **Batch size:** Number of concurrent sequences

Optimization Targets

Different use cases require different optimizations:

Interactive chat (latency-critical):

- Minimize TTFT and ITL
- Smaller batch sizes acceptable
- Focus on responsiveness

Optimization: Throughput and Cost

Batch processing (throughput-critical):

- Maximize tokens/sec
- Larger batch sizes
- Latency less critical

Cost optimization:

- Maximize throughput per GPU
- Balance quality vs quantization
- Efficient memory usage

Summary

Key Takeaways

Part I: Decoder-Only Transformers:

1. **Decoder-only** transformers use causal masking for autoregressive generation
2. **Training objective**: Predict next token given previous tokens
3. **Causal mask**: Lower triangular matrix prevents seeing future
4. **GPT-2 block**: LayerNorm $\rightarrow$ Attention $\rightarrow$ Residual $\rightarrow$ LayerNorm $\rightarrow$ FFN $\rightarrow$ Residual
5. **Generation**: Multiple strategies (greedy, temperature, top-k, top-p)
6. **vs BERT**: Causal vs bidirectional attention enables generation vs understanding

Key Takeaways (cont.)

Part II: Training LLMs:

1. **BPE tokenization**: Balances vocabulary size and coverage
2. **Pretraining**: Massive scale, predicts next token, learns language
3. **Instruction tuning**: Supervised learning on (prompt, response) pairs
4. **RLHF**: Three stages (rankings $\rightarrow$ reward model $\rightarrow$ PPO)
5. **MoE**: Scale capacity by activating subset of parameters
6. **Reasoning models**: RL training emphasizes multi-step thinking

Key Takeaways (cont.)

Part III: Optimizing Inference:

1. **KV cache**: PagedAttention for memory efficiency
2. **Quantization**: FP8/INT8 for weights and cache
3. **Batching**: Continuous batching for high utilization
4. **Runtimes**: Use optimized engines (vLLM, TensorRT-LLM)

Impact: 10-20$\times$ improvement in throughput and cost efficiency

Learning Objectives Review

After this lecture, you should be able to:

- Distinguish transformer architecture variants and explain GPT-2's design
- Describe the complete LLM training pipeline (pretraining $\rightarrow$ SFT $\rightarrow$ RLHF)
- Explain BPE tokenization, MoE architecture, and reasoning models
- Explain the two phases of LLM inference (prefill and decode)
- Understand why KV cache is critical and how PagedAttention works
- Compare quantization strategies and batching approaches

References

Architecture and Training Papers

1. Vaswani et al. (2017). **Attention Is All You Need**. NeurIPS.
2. Radford et al. (2018). **Improving Language Understanding by Generative Pre-Training**. OpenAI.
3. Radford et al. (2019). **Language Models are Unsupervised Multitask Learners**. OpenAI.
4. Sennrich et al. (2016). **Neural Machine Translation of Rare Words with Subword Units**. ACL.
5. Ouyang et al. (2022). **Training language models to follow instructions with human feedback**. NeurIPS.
6. Schulman et al. (2017). **Proximal Policy Optimization Algorithms**. arXiv:1707.06347.
7. Fedus et al. (2022). **Switch Transformers: Scaling to Trillion Parameter Models**. JMLR.

Inference Optimization Paper: Clickable Links

1. Efficient Memory Management for Large Language Model Serving with PagedAttention
2. GPTQ: Accurate Post-Training Quantization for GPT
3. AWQ: Activation-aware Weight Quantization
4. FP8 Formats for Deep Learning
5. Flash Attention: Fast and Memory-Efficient Exact Attention

Further Resources (Clickable Links)

Interactive visualizations:

- Transformer Explainer
- The Illustrated GPT-2
- LLM Visualization
- The Illustrated Transformer
- Illustrating Reinforcement Learning from Human Feedback

Further Resources (Clickable Links cont.)

Implementations:

- Hugging Face Transformers
- nanoGPT - Minimal GPT implementation
- GPT-2 Repository
- Hugging Face PEFT - Parameter-efficient fine-tuning
- Hugging Face TRL - Transformer reinforcement learning
- DeepSpeed - Distributed training
- vLLM - Fast LLM serving
- NVIDIA TensorRT-LLM

Datasets:

- The Pile - Large-scale pretraining dataset
- Alpaca - Instruction tuning dataset